SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

Duration :60 Mins
On Class
Total Marks:50

QUESTIONS:5

Annexure A

DESIGN TASK

Code of Conduct:

I M.VAISHNAV(192524125)*(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	30	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	10	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation

R. VAISHNAV
192524251

CO2 - AT1 (Compiler Design)

Q1. LL (1) Predictive parser Design:

given grammar,

S -> ABD
A -> a|Db|ε
B -> gD|dA|ε
D -> e|f

(i) First and follow sets

First Sets

First (D)
D -> e|f
first (D) = {e, f}

First (A)
A -> a|Db|ε
First (Db) = First(D) = {e, b}
First (A) = {a, e, f, ε}

First (B)
B -> gD|dA|ε
First (B) = {g, d, ε}

First (S)
S -> ABD
First (A) = {a, e, fε}

Since A can produce ε, check B.

First (B) = {g, d, ε}

Since B can also produce ε, check D.

First (D) = {e, f}

∴ First (S) = {a, e, f, g, D}

Start Symbol:-

Follow (S) = {$}

From S → ABD

For A :-

Follow (A) contains First(BD)

first (BD) = {g, d, e, f}

Follow (A) = {g, d, e, f}

For B :-

first (D) = {e, f}

follow (B) = {e, f}

For D :-

Follow (D) contains Follow (S)

Follow (D) = {$}

From A → Db

Follow (D) = {b, $}

From B → gD

Follow (D) contains Follow (B)

Follow (D) = {b, $, e, f}

From B → dA

Follow (A) contains follow (B)

follow (A) = {g, d, e, f}

Final follow sets;

Follow (S) = {$}
Follow (A) = {g, d, e, f}
Follow (B) = {e, f}
Follow (D) = {b, $, e, f}

Predictive parsing Table :-

Non-terminal	a	b	d	e	f	g	$
S	S → ABD	-	S → ABD	S → ABD	S → ABD	S → ABD	-
A	A → a	-	A → ε	A → Db	A → Db	A → ε	-
B	-	-	B → dA	B → ε	B → ε	B → gD	-
D	-	-	-	D → e	D → f	-	-

(iii) verification of LL(1) property First / First conflict Analysis

For A :-

First(a) = {a}

First(Db) = {e, f}

First(ε) = {ε}

Since A contains ε,

First(A) − {ε} = {a, e, f}

Follow(A) = {g, d, e, f}

Intersection :-

{e, f}

≠ φ

Conflict exists

Parsing Table conflict :-

For entries M[A, e] and M[A, f] :-

$A \to Db$

and

$A \to \epsilon$

both appear.

Hence multiple entries occur.

(iv) Demonstration of predictive parsing

aed \$ → Input String

Derivation :-

$S \to ABD \to aBD \to aD \to ae$

Input = ae\$

Parsing Table :-

Stack	Input	Action
\$S	ae\$	$S \to ABD$
\$DBA	ae\$	$A \to a$
\$DBa	ae\$	Match a
\$DB	e\$	$B \to \epsilon$
\$D	e\$	$D \to e$
\$e	e\$	Match e
\$	\$	Accept

Input string ae is successfully accepted

(v) Design Recommendations :-

Can the grammar be used directly ?

No.

It cannot be used directly for LL(1) parsing.